

Product Requirements Document (PRD)

Project Name: BonPlan.ai
Document Status: Approved for Software Requirements Specification (SRS)
Product Vision: Tell us When. We Tell the How.

1.0 Executive Summary

BonPlan.ai is a constraint-based AI travel routing and scheduling engine. Unlike standard generative AI travel tools that produce static, unverified itineraries, BonPlan.ai utilizes an agentic architecture (ReAct loop) to ingest user-defined logistical constraints ("Smart Anchors") and autonomously generate geographically verified, conflict-free travel schedules around them.

2.0 Problem Statement & Market Gap

Current AI travel planners suffer from "Context Amnesia" and "Geographical Blindness." They operate as text generators rather than logistical calculators. When presented with hard constraints (e.g., a pre-booked 2:00 PM flight and an 8:00 PM non-refundable dinner reservation), existing tools frequently overwrite the constraints, fail to calculate real-world transit times, or suggest geographically impossible sequences. BonPlan.ai solves the "routing and fitting" problem by treating user anchors as immutable database states and mathematically calculating the available whitespace before calling external APIs to verify recommendations.

3.0 Target Personas

Persona	Primary Characteristic	Core Pain Point	Primary BonPlan.ai Solution
Type-A Planner	Books high-value reservations months in advance; maintains strict schedules.	Standard AI tools overwrite their prized reservations and	Smart Anchors: Immutable schedule locks with automated transit buffers.

		ignore transit buffers.	
Group Trip Coordinator	Manages complex itineraries for multiple stakeholders with conflicting interests.	Building consensus and adjusting plans based on group feedback is tedious.	Multiplayer Collaborative Mode: Real-time canvas with democratic activity voting.
Dynamic Explorer	Adapts plans on the fly based on weather, crowds, or spontaneous interests.	Static itineraries break completely when one variable (e.g., weather) changes.	Specific Regenerations: Surgical, isolated schedule swaps without cascading failures.
Bleisure Traveler	Mixes corporate obligations (conferences/meetings) with localized tourism.	Needs to know the exact feasibility of activities within strict 2-4 hour gaps.	Constraint Gap-Filling: Maximizes efficiency in localized geographic clusters.
Blank Canvas Traveler	High intent to travel, but zero local knowledge or pre-existing plans.	Severe decision fatigue from researching unstructured travel blogs and maps.	Trip For All: Conversational onboarding that generates a baseline, logically sound itinerary.

4.0 Core Functional Requirements (Features)

FR-1: Smart Anchors (Constraint Locking)

The system must allow users to define immutable schedule blocks (flights, reservations). The AI orchestration layer is prohibited from modifying or deleting these blocks during itinerary generation.

FR-2: Dynamic Whitespace Calculation

The backend must calculate viable "free time" between Smart Anchors, automatically deducting localized transit times and check-in buffers before passing the time constraints to the LLM.

FR-3: Smart Adjustments (Auto-Shuffling)

A drag-and-drop React frontend must allow users to extend or compress an activity's duration. The backend must cascade the time changes to subsequent events, stopping the cascade and alerting the user if a Smart Anchor is violated.

FR-4: Surgical Activity Swaps (Specific Regenerations)

Users can highlight a single event block and request an alternative. The agentic loop must query external APIs for a replacement that fits the exact geographic and temporal boundaries of the deleted event, leaving the rest of the timeline untouched.

FR-5: Customized Pacing Algorithms

Users must be able to define the itinerary density (e.g., "Relaxed" vs. "Action-Packed"), which directly dictates the minimum rest intervals the AI injects between scheduled events.

FR-6: Live-State Concierge Integration

Post-generation, the system must ping external APIs to provide dynamic updates (weather shifts, traffic routing, booking statuses) and offer proactive, single-click schedule rerouting.

FR-7: Conversational Context (Trip for All)

The system must allow speaking to it exactly how an user would speak to a human travel agent. After just describing the vibe, whether it's a solo backpacking trek, a family-friendly holiday, or an adventurous weekend with friends, the AI must generate the recommendations to perfectly match the context.

5.0 System Architecture & Tech Stack

Component	Technology	Purpose
Frontend Application	React, dnd-kit	Render the interactive UI, handle optimistic state updates during drag-and-drop actions.
Backend API	FastAPI (Python)	High-concurrency routing, state management, and orchestration of the Agentic loop.
Database	PostgreSQL	Relational mapping of Trips, Days, Events, and strict enforcement of the <code>is_anchor</code> boolean.
AI Orchestration	LangChain / LlamaIndex	Management of the ReAct (Reasoning and Acting) loop and tool-calling framework.
External APIs	Google Maps/Places, Amadeus	Real-world grounding for transit times, business operating hours, and flight logistics.

6.0 Technical Risks & Mitigation

Risk Area	Architectural Challenge	Proposed Mitigation Strategy
State Syncing & Race Conditions	Users rapidly adjust the visual timeline before the backend completes transit recalculations.	Implement strict optimistic UI updates with a synchronized backend queue; trigger visual warnings if backend validation fails.
Agentic Latency	Sequential API calls (LLM → Places → Maps → LLM) causing 15-30 second generation delays.	Implement Server-Sent Events (SSE) or WebSockets to stream the agent's "thought process" (e.g., "Verifying transit...") to the UI to maintain engagement.
Infinite Retrieval Loops	LLMs failing to find activities that fit strict constraints and endlessly querying external APIs.	Enforce rigid <code>max_iterations</code> limits on the LangChain agent; trigger a hard-coded "Negotiation Mode" prompt to ask the user to loosen constraints.
Domino Effect Resolution	Extending one activity pushes subsequent activities into a locked Smart Anchor.	Develop a localized shifting algorithm that prompts the user to either compress transit buffers or explicitly drop the lowest-priority event.
Geographic Hallucinations	The LLM fabricating locations or ignoring closed hours.	Enforce JSON schema validation that mandates a verified Google Places ID and an <code>open_now == true</code> flag.

